

Постановка задачи.

1. Ввести 10-15 целых чисел и построить из них с помощью указателей бинарное дерево поиска.
2. Обойти дерево прямым, симметричным и обратным способами. Вывести результаты обходов.
3. Реализовать процедуры поиска, вставки и удаления элементов бинарного дерева поиска с последующим отображением дерева.

Листинг программы:

```
#include <iostream>
#include <iomanip>
#include <string>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

// Генерация случайного целого числа в диапазоне
int randInt(int min, int max) { return rand() % (max - min + 1) + min; }

// Валидация ввода целого числа
void validateInt(int& a, string varName)
{
    string sA = "";
    do
    {
        try
        {
            if (varName != "")
                cout << "Введите " << varName << ": ";
            cin >> sA;
            a = stoi(sA);
            break;
        }
        catch (invalid_argument ex) {
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            cout << "Введено слишком большое число" << endl;
        }
    } while (true);
}

// Структура узла дерева
struct treeNode
{
    int field; // поле данных
    struct treeNode* left; // указатель на левое поддерево
    struct treeNode* right; // указатель на правое поддерево
};

// Вывод дерева в прямом порядке (префиксная форма)
void displayInOrderTree(treeNode* tree)
{
    if (tree != NULL)
    {
        cout << tree->field << " ";
        displayInOrderTree(tree->left);
        displayInOrderTree(tree->right);
    }
}
```

```

// Вывод дерева в симметричном порядке (инфиксная форма)
void displaySortedTree(treeNode* tree)
{
    if (tree != NULL)
    {
        displaySortedTree(tree->left);
        cout << tree->field << " ";
        displaySortedTree(tree->right);
    }
}

// Вывод дерева в обратном порядке (постфиксная форма)
void displayInPostfixTree(treeNode* tree)
{
    if (tree != NULL)
    {
        displayInPostfixTree(tree->left);
        displayInPostfixTree(tree->right);
        cout << tree->field << " ";
    }
}

// Добавление узла в дерево
struct treeNode* addNode(int x, treeNode* tree)
{
    if (tree == NULL)
    {
        tree = new treeNode;
        tree->field = x;
        tree->left = NULL;
        tree->right = NULL;
    }
    else
    {
        if (x < tree->field)
            tree->left = addNode(x, tree->left);
        else
            tree->right = addNode(x, tree->right);
    }
    return(tree);
}

// Поиск узла в дереве
treeNode* findNode(int x, treeNode* tree)
{
    if (tree == NULL)
        return NULL;
    if (tree->field == x)
        return tree;

    if (x <= tree->field)
    {
        if (tree->left != NULL)
            return findNode(x, tree->left);
        else
            return NULL;
    }
    else
    {
        if (tree->right)
            return findNode(x, tree->right);
        else
            return NULL;
    }
}

```

```

    }
}

// Обертка для более удобного поиска
void findHandler(treeNode* tree) {
    int num;
    validateInt(num, "искомое число");
    struct treeNode* tn1 = findNode(num, tree);
    if (tn1)
        cout << tn1->field << endl;
    else
        cout << "Числа " << num << " в дереве нет" << endl;
}

void deleteNode(int key, treeNode* tree)
{
    treeNode* parent = NULL;
    while (tree && tree->field != key)
    {
        parent = tree;
        if (tree->field > key)
        {
            tree = tree->left;
        }
        else
        {
            tree = tree->right;
        }
    }
    if (!tree)
        return;
    if (tree->left == NULL)
    {
        // Вместо tree подвешивается его правое поддерево
        if (parent && parent->left == tree)
            parent->left = tree->right;
        if (parent && parent->right == tree)
            parent->right = tree->right;

        delete tree;
        return;
    }
    if (tree->right == NULL)
    {
        // Вместо tree подвешивается его левое поддерево
        if (parent && parent->left == tree)
            parent->left = tree->left;
        if (parent && parent->right == tree)
            parent->right = tree->left;

        delete tree;
        return;
    }

    // У элемента есть два потомка, тогда на место элемента поставим
    // наименьший элемент из его правого поддерева
    treeNode* replace = tree->right;
    while (replace->left)
        replace = replace->left;
    int replace_value = replace->field;
    deleteNode(replace_value, tree);
    tree->field = replace_value;
}

```

```

// Очистка дерева
void freeMemory(treeNode* tree)
{
    if (tree != NULL)
    {
        freeMemory(tree->left);
        freeMemory(tree->right);
        delete tree;
    }
}

int main()
{
    system("chcp 1251");
    system("cls");

    // Инициализация корня
    struct treeNode* root = 0;

    cout << "Выберите пункт меню\n1. Ввести узлы вручную\n2. Заполнить случайными
числами в диапазоне\n3. Заполнить заготовленными числами\n";
    int menu;
    validateInt(menu, "");
    switch (menu)
    {
    case 1:
    {
        int a, n;
        cout << "Сколько чисел будет вводиться? [10, 15]\n";
        do
        {
            validateInt(n, "");
        } while (n < 10 || n > 15);

        for (int i = 0; i < n; i++)
        {
            cout << "Введите узел " << i + 1 << ": ";
            validateInt(a, "");
            root = addNode(a, root);
        }
    }
    break;
    case 2:
    {
        int min, max;
        cout << "Введите минимальное генерируемое число: \n";
        validateInt(min, "");
        cout << "Введите максимальное генерируемое число: \n";
        validateInt(max, "");
        for (int i = 0; i < 15; i++)
            root = addNode(randInt(min, max), root);
        break;
    }
    default:
    {
        root = addNode(4, root);
        root = addNode(3, root);
        root = addNode(5, root);
        root = addNode(1, root);
        root = addNode(7, root);
        root = addNode(8, root);
        root = addNode(6, root);
        root = addNode(2, root);
        break;
    }
}

displayInOrderTree(root);

```

```

cout << "\n";
displaySortedTree(root);
cout << "\n";
displayInPostfixTree(root);
cout << "\n";

int a;
validateInt(a, "элемент, который надо добавить в дерево");
addNode(a, root);
displaySortedTree(root);
cout << "\n";

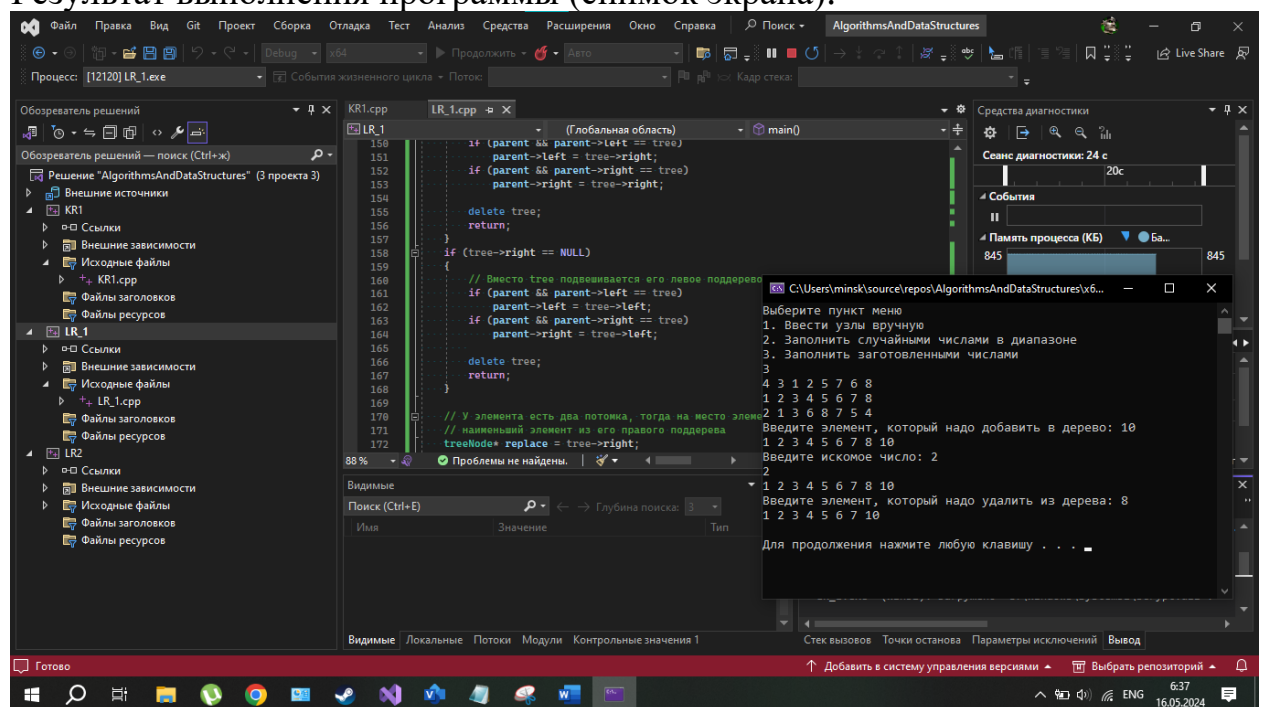
findHandler(root);
displaySortedTree(root);
cout << "\n";

validateInt(a, "элемент, который надо удалить из дерева");
deleteNode(a, root);
displaySortedTree(root);
cout << "\n";

cout << "\n";
freeMemory(root);
system("pause");
return 0;
}

```

Результат выполнения программы (снимок экрана):



Ответы на контрольные вопросы:

1. Дайте определение абстрактному типу данных «дерево».
Дерево – структура данных, представляющая собой древовидную структуру в виде набора связанных узлов.
2. Перечислите названия узлов, встречающихся в дереве, и дайте определение каждому из них.
Корень – элемент, находящийся первым и не имеющий предшественников.
Узел – любой элемент дерева.
Лист – элемент, не имеющий потомков.
3. Как определить, какие узлы в дереве расположены слева от данного узла n , а какие – справа?
Узел n содержит указатель на левый и правый по отношению к нему узел.
4. Дайте определение бинарному дереву и бинарному дереву поиска.
Бинарное дерево — это конечное множество элементов, которое либо пусто, либо содержит элемент (корень), связанный с двумя различными бинарными деревьями, называемыми левым и правым поддеревьями.

Бинарное (двоичное) дерево поиска – это бинарное дерево, для которого выполняются следующие дополнительные условия (свойства дерева поиска):
 - оба поддерева – левое и правое, являются двоичными деревьями поиска;
 - у всех узлов левого поддерева произвольного узла X значения ключей данных меньше, чем значение ключа данных самого узла X ;
 - у всех узлов правого поддерева произвольного узла X значения ключей данных не меньше, чем значение ключа данных узла X .
5. Назовите три основных способа обхода бинарных деревьев и укажите порядок следования вершин, соответствующий каждому из обходов.
 - Прямой (сверху вниз);
 - Обратный (снизу вверх);
 - Слева направо (симметричный).